

THEORY ASSIGNMENT-1

SUBJECT:

OOP

DEPARTMENT:

BSAI

ROLL NO:

SP26-BAI-037

SUBMITTED BY:

Sheraz Shahbaz

SUBMITTED TO:

Sir Shahid Bhatti



Object Oriented Analysis of a Smart Campus Allocation System

Task 1 — Stakeholder and Requirement Analysis

1.1 Stakeholders and their needs

Stakeholder	Main needs
Students	Fair access to classrooms, project spaces and other suitable rooms.
Faculty members	Reliable access for scheduled classes and urgent academic activities.
Administration	Good utilization of available rooms while following university policies.
Maintenance staff	Ability to immediately block unsafe or maintenance rooms.
Security staff	Traceable access and reservation records.
Disability support staff	Accessible spaces remain available for users who need them.

1.2 Requirement Conflict:

- **Fair access vs. faculty priority:**
Students expect fair access, while faculty may need priority for teaching and urgent academic activity.
Assumption: scheduled teaching receives higher priority than ordinary activities.
- **Room utilization vs. safety:**
Administration wants high utilization, but maintenance must block unsafe rooms immediately.
Assumption: safety takes priority; affected reservations are cancelled or relocated.
- **Traceability vs. privacy:**
Security needs records, while users expect private information to remain protected.
Assumption: authorized staff can access audit information while ordinary users see only necessary information.

1.3 Facts and assumptions

Facts given in the scenario:

- The system will allocate classrooms, laboratories, seminar rooms, and project spaces.
- Different activities can have different capacity, equipment, and approval requirements.

- Students expect fair access.
- Faculty may require priority for teaching and urgent academic activities.
- Administration wants high room utilization.
- Maintenance staff must be able to block unsafe rooms.
- Security staff need traceable access records.
- A person may have more than one role.
- Roles and permissions can change.
- Emergency events may cancel or relocate confirmed reservations.
- The system must retain enough information for auditing and notify affected users.
- Some rules, such as the exact meaning of fairness, are not completely defined.

My assumptions

- Scheduled teaching has priority over ordinary activities.
- Unsafe rooms cannot be allocated until maintenance clears them.
- When an emergency affects a reservation, the system will first try to relocate it if a suitable space is available.
- If no suitable alternative exists, the reservation will be cancelled.
- Every cancellation or relocation caused by an emergency will create an audit record.
- Ordinary users will not be allowed to change reservation status directly.

These assumptions make the incomplete requirements more specific without changing the main scenario.

Task 2 — Object-Oriented Conceptual Model

2.1 Proposed classes

I propose the following eight classes:

1. Person

Responsibility: Represents a person who interacts with or is affected by the system.

Attributes:

- personID
- name
- contactInformation

Operations:

- requestReservation()
- viewAvailability()

2. Role

Responsibility: Represents a role assigned to a person and the permissions associated with that role.

Attributes:

- roleName
- permissions

Operations:

- addPermission()
- removePermission()

3. Building

Responsibility: Represents a university building containing learning spaces.

Attributes:

- buildingID
- buildingName

Operations:

- closeBuilding()
- getAvailableSpaces()

4. Space

Responsibility: Represents a classroom, laboratory, seminar room or project space that can be allocated.

Attributes:

- spaceID
- capacity
- status

Operations:

- blockSpace()
- checkAvailability()

5. Reservation

Responsibility: Represents a request or confirmed allocation of a space for a particular activity and time.

Attributes:

- reservationId
- startTime
- endTime
- status

- **Operations:**

- confirm()
- cancel()
- relocate()

6. Equipment

Responsibility: Represents equipment that may be available in or assigned to a space.

Attributes:

- equipmentId
- equipmentType

- availabilityStatus

Operations:

- allocate()
- release()

7. Notification

Responsibility: Represents messages sent to users when important reservation events occur.

Attributes:

- notificationId
- message
- dateTime

Operations:

- send()
- markAsRead()

8. AuditRecord

Responsibility: Stores important system events so that actions can be traced later.

Attributes:

- recordId
- action
- timestamp

Operations:

- createRecord()
- viewRecord()

2.2 UML class relationships

```
Building 1 ----- 1..* Space
Space 1 ----- 0..* Reservation
Person 1 ----- 0..* Reservation
```

```
Person 1 ----- 1..* Role
Reservation 0..* ----- 0..* Equipment
Reservation 1 ----- 0..* Notification
Reservation 1 ----- 0..* AuditRecord
```

The relationships show that a building contains spaces, spaces can have reservations over various times, people make reservations and can have multiple roles, and reservations can produce notifications and audit records. Equipment can be associated with reservations.

2.3 Responsibility separation

I have intentionally divided the system into different classes instead of creating one large class.

For example:

- Person deals with the user.
- Role deals with permission.
- Space deals with room-related information.
- Reservation deals with reservation state.
- Building deals with building-level events such as closure.
- Notification deals with communication.
- AuditRecord deals with traceability.
- Equipment deals with resources.

This separation makes the model easier to understand and modify. If the notification process changes, there is no need to redesign the entire reservation class.

Task 3 — Abstraction, Encapsulation, and Invariants

3.1 Class Reservation

Abstraction: Represents requester, space, time and booking status.

Hidden: status and internal approval/cancellation information.

Controlled operations: confirm(), cancel(), relocate().

3.2 Class Space:

Abstraction: Represents a learning space without unnecessary physical detail.

Hidden: room status, maintenance state, and allocation information.

Controlled operations: blockSpace(), checkAvailability().

3.3 Class Person

Abstraction:

Represents a person without requiring a separate class for every university role.

Hidden: private contact and permission information.

Controlled operations: requestReservation(), viewAvailability().

3.4 Invariants:

- **Invariant 1**

A reservation cannot be confirmed for a blocked or closed space.

This prevents unsafe rooms from being allocated.

- **Invariant 2**

Two confirmed reservations cannot overlap for the same space.

This prevents two users from being assigned the same room at the same time.

- **Invariant 3**

A reservation must have a valid start time before its end time.

This prevents meaningless or invalid reservation periods.

Invariant 4

A closed building cannot provide a space for a new confirmed reservation.

This is especially important for my Emergency Continuity condition.

When a building is closed because of an emergency, the system should prevent new allocations and identify affected existing reservations.

- **Role of encapsulation:**

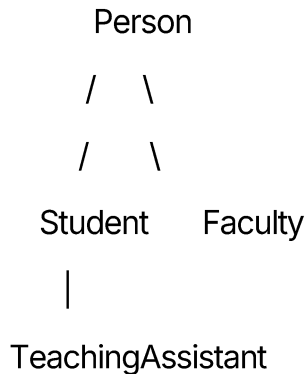
- Encapsulation helps protect these invariants because important data should not be freely changed from outside the object.
- For example, instead of allowing:
 - reservation.status = "Confirmed"
 - the design should use an operation such as:
 - reservation.confirm()
- The operation can first check availability, space status and other required conditions. Therefore, the object has more control over maintaining a valid state.

Task 4 — Alternative Designs and Decision

The system must support people who can have more than one role and whose roles may change during the semester. I considered two designs.

Alternative 1: Inheritance

One possible design is:



In this approach, different types of people are represented as subclasses of Person.

Advantages

Inheritance is easy to understand and works well when categories are fixed. Common attributes and operations can be placed in the parent class.

Problems

The scenario says that a person may have more than one role. A person could be a student and teaching assistant at the same time. They may also change roles later.

Inheritance can become complicated if every possible combination needs a separate subclass.

For example, creating classes such as:

Student

Faculty

StudentTeachingAssistant

FacultyAdministrator

StudentAdministrator

would make the model unnecessarily complicated.

Alternative 2: Composition using Role objects

In the second design, a Person contains one or more Role objects.

Person

|

+---- StudentRole

|

+---- TeachingAssistantRole

A person's roles can be added or removed without changing the person's class.

Advantages

- Supports multiple roles naturally.
- Roles can change without changing the person's type.
- New roles can be added more easily.
- Permissions can be associated with roles.
- Reduces duplicate behavior between different types of users.

Comparison

Criterion	Inheritance	Composition / Role Objects
Multiple roles	More difficult	Naturally supported
Changing roles	May require type changes	Roles can be added/removed
Extensibility	New subclasses may be needed	New role objects can be introduced
Duplicate behavior	Can occur across subclasses	Common behavior stays in Person
Coupling	More tied to hierarchy	More flexible

Decision

For this system, I would use composition with role objects. The main reason is that the requirements specifically allow one person to have more than one role and allow permissions to change during the semester. A role-based composition model matches these requirements more directly.

It also works well with emergency continuity because permissions can be changed without redesigning the person's basic identity.

Task 5 — Behaviour and Edge Cases

5.1 Reservation life cycle

Pending → Confirmed → Completed

Pending → Rejected

Confirmed → Cancelled

Confirmed → Relocated → Completed

A reservation starts as Pending. If availability and approval conditions are satisfied, it becomes Confirmed. It can later become Completed, Cancelled or Relocated.

5.2 Two simultaneous requests

If two users request the same space for overlapping times, the system checks existing reservations and applies priority rules. If both have equal priority, I assume the earlier valid request received by the system is considered first. Reservation controls booking state while Space provides availability information.

5.3 Maintenance block after confirmation

If maintenance blocks a space after confirmation, the reservation cannot continue in an unsafe space. The system searches for a suitable alternative. If one exists, the reservation is relocated; otherwise it is cancelled. The user is notified and an audit record is created.

5.4 Emergency cancellation or relocation

This case is particularly important because my registration-number condition is **Emergency Continuity**.

If an entire building is closed, many reservations may be affected at once.

The process would be:

Emergency

↓

Building closed

↓

Find spaces in building



Find affected reservations



Check alternative spaces



Relocate where possible



Cancel where relocation is impossible



Notify affected users



Create audit records

The Building object is responsible for representing the building closure. The affected Reservation objects then change state. Notification communicates the result to users, while AuditRecord keeps evidence of what happened.

Responsibility placement

Decision	Main object	Reason
Space available or blocked	Space	It owns the space's current state.
Reservation status	Reservation	It represents the reservation life cycle.
Building closure	Building	The closure affects the whole building.
User notification	Notification	Communication is its responsibility.
Audit information	AuditRecord	It stores traceable system events.
Permission to perform actions	Role	Permissions depend on the user's assigned role.

This division prevents one object from becoming responsible for every decision.

TASK- 6 Independent Exploration: Immutable Value Objects :

An immutable value object is an object whose value cannot be changed after it has been created. If a different value is needed, a new object is created instead. This approach can make a system easier to understand and can reduce accidental changes to important information.

In the Smart Campus Allocation System, a **TimeSlot** can be modeled as an immutable value object. A TimeSlot can contain a start time and an end time for a reservation. Once a reservation receives its TimeSlot, the start and end times should not be changed directly. If the reservation needs a different time, a new TimeSlot can be created.

This improves the model because reservation timing is important for avoiding conflicts. For example, if a TimeSlot could be changed unexpectedly, a confirmed reservation might overlap with another reservation. Keeping the TimeSlot immutable makes its value more reliable after creation.

The concept also supports encapsulation because the internal values are protected from unwanted modification. Instead of allowing other objects to directly change the start or end time, the system can create a new valid TimeSlot when necessary.

Therefore, an immutable TimeSlot is useful in this system because it keeps important reservation information stable, reduces accidental changes, and makes the reservation model easier to reason about.

References

1. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.
2. Oracle. (n.d.). *Object-Oriented Programming Concepts: Encapsulation*. Java Documentation.

Generative AI Disclosure :

I used a generative AI tool to help understand the assignment requirements, organize object-oriented analysis, and improve the clarity of the written explanation. I reviewed the resulting material and remained responsible for the final content, assumptions, class design, and decisions presented in this assignment.